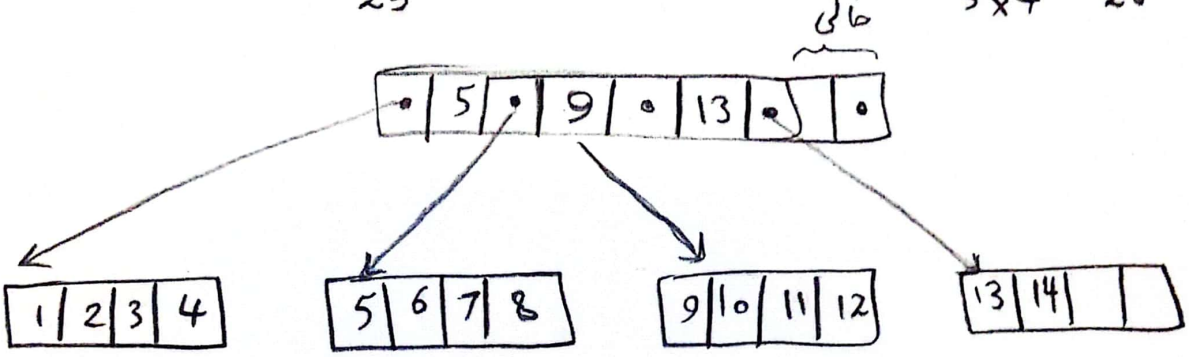
DB - HW7 ۸۱۰۱۰۱۴۹۲ علیرضا کریمی

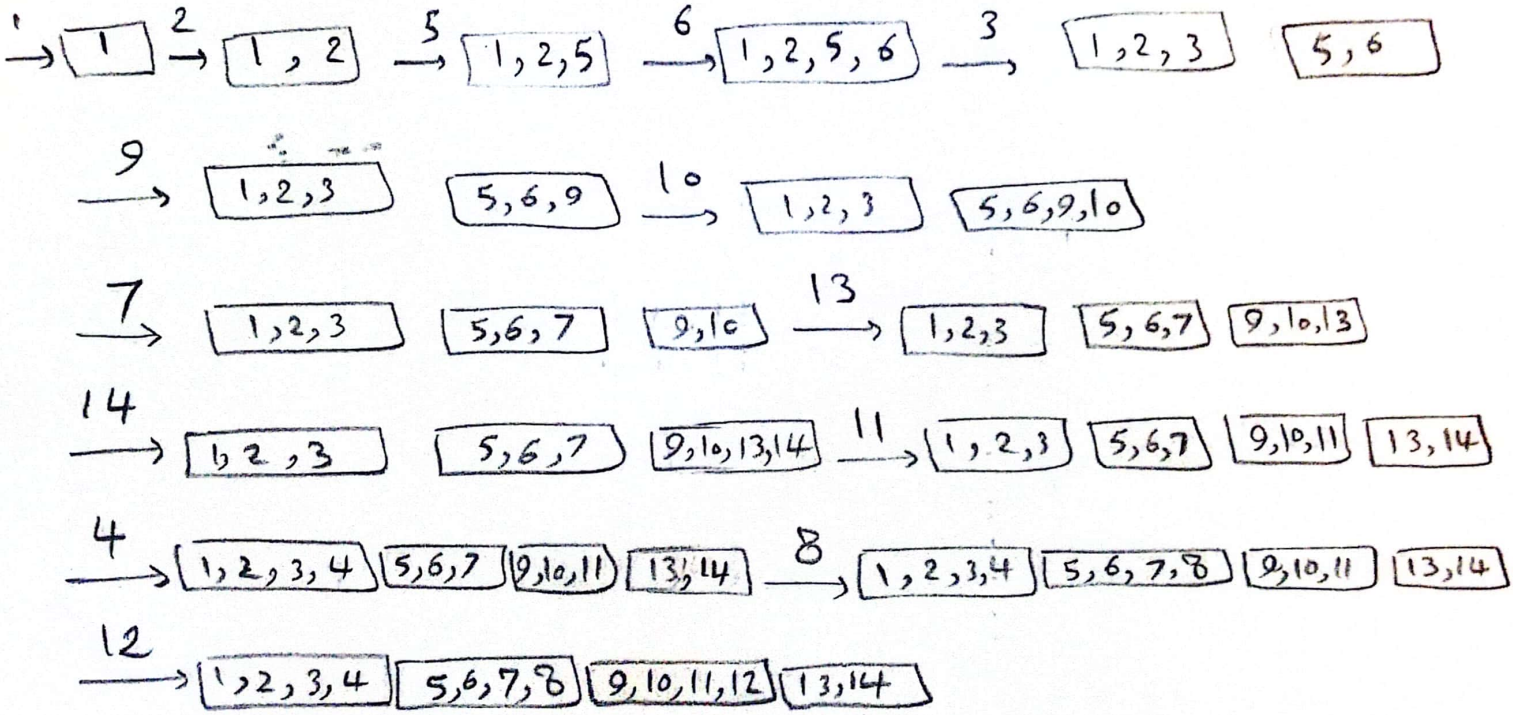
۱) space utilization: $\frac{21}{25} = ۸۴\%$ ⟹ بدون حساب کردن حجم پوینتر $\frac{14+3}{5\times4} = \frac{17}{20}$

الف)



ب) ترتیب:

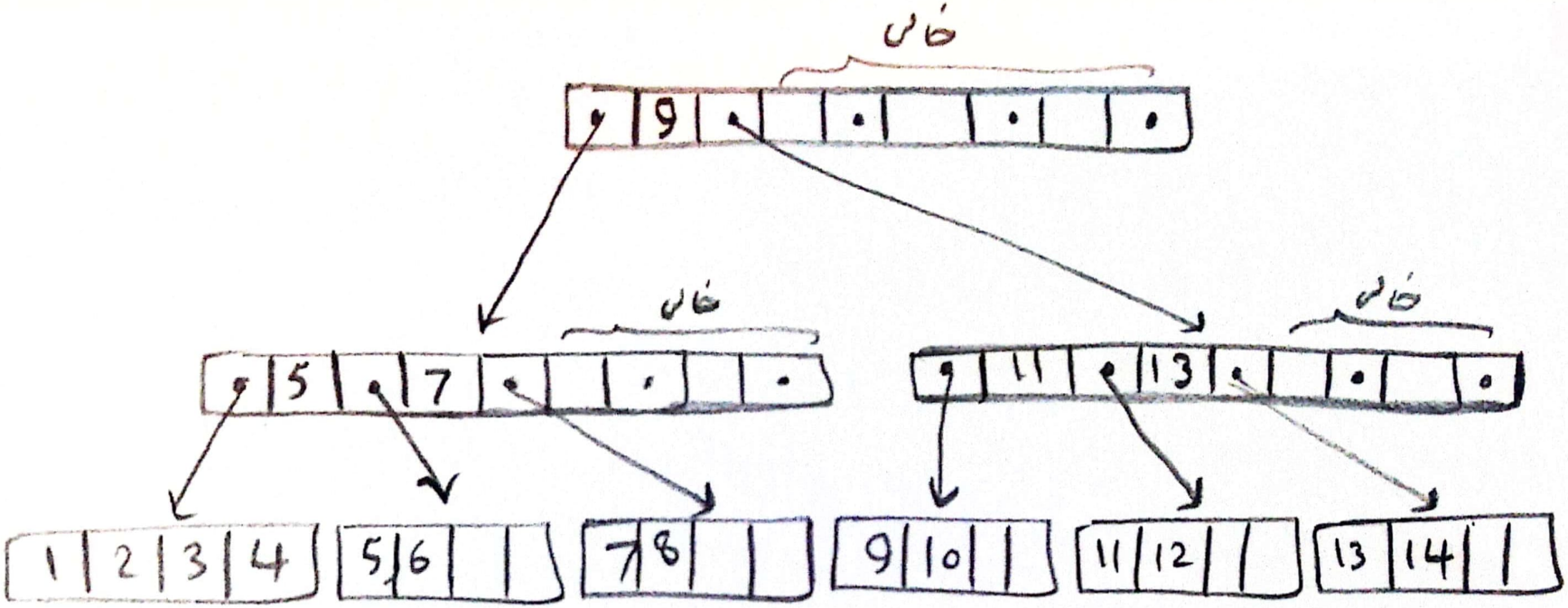
⟶ 1, 2, 5, 6, 3, 9, 10, 7, 13, 14, 11, 4, 8, 12



البته اگر bulk loading می کردیم اعداد ۱ تا ۱۴ را به ترتیب، به همین نتیجه می رسیدیم.

ادامه ۱۱

ج)



$$\Longrightarrow \text{space utilization} = \frac{4 + 2\times5 + 2\times5 + 3}{6\times4 + 2\times9 + 9} = \frac{27}{51}$$

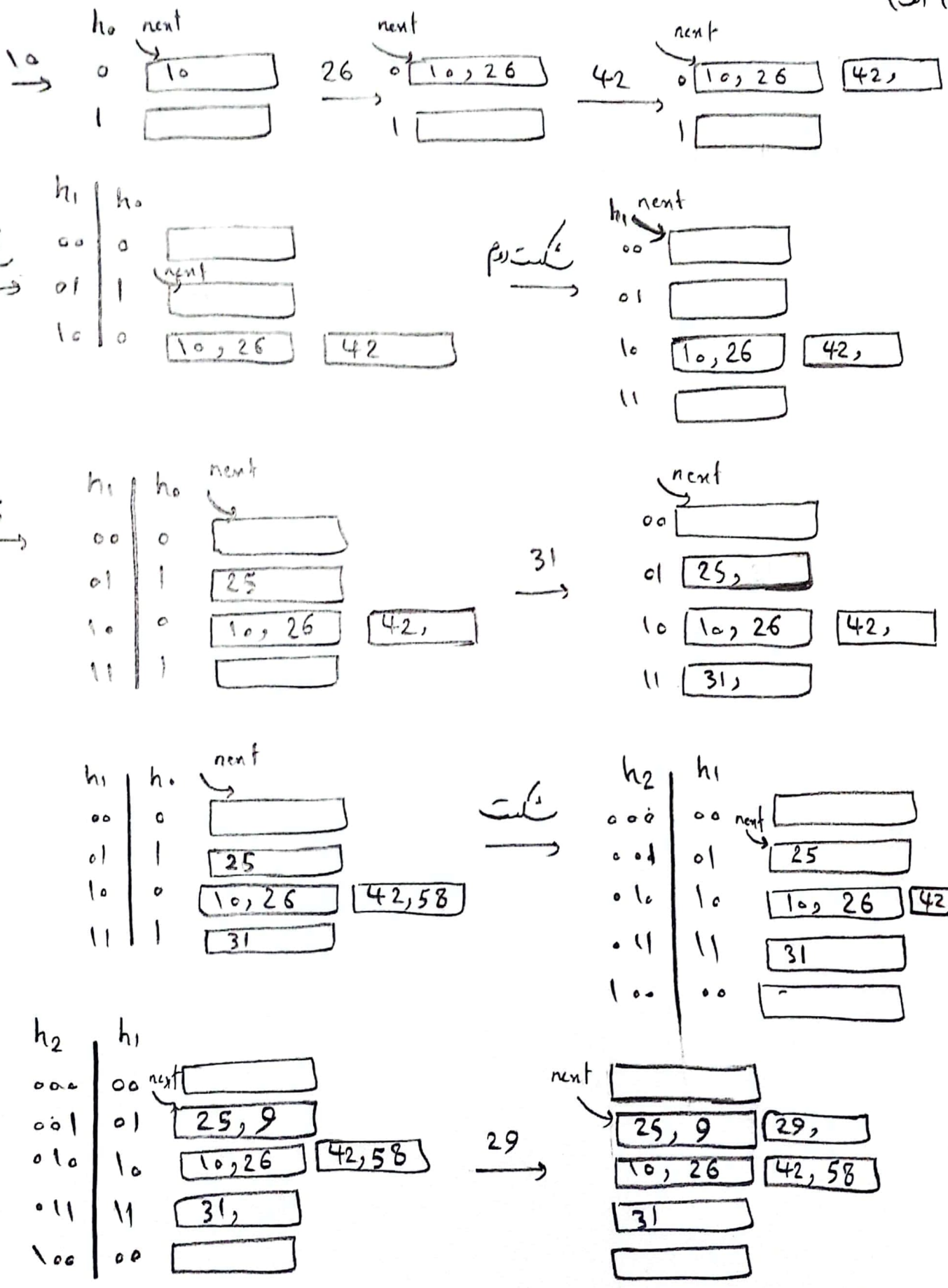
(internal nodes: 2×5; root: 3; internal nodes: 2×9; root: 9)

بدون حساب کردن حجم پوینتر

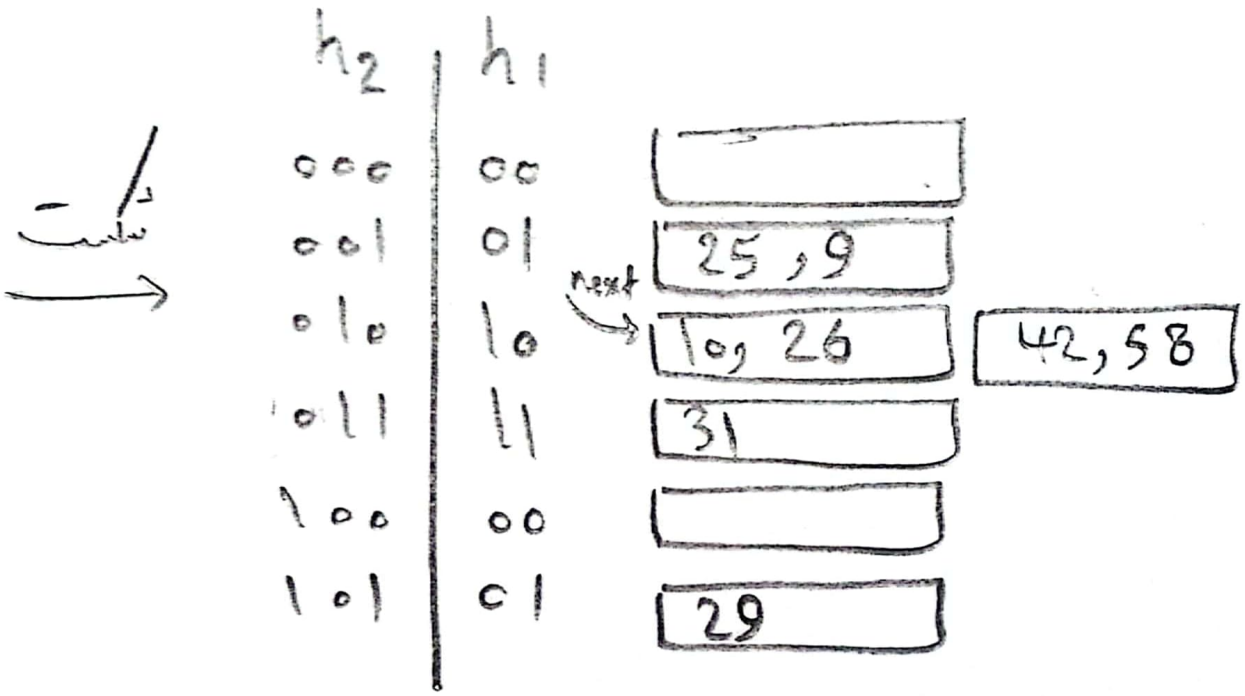
$$\Longrightarrow \frac{14+4+1}{9\times4} = \frac{19}{36}$$

د) ترتیب اضافه کردن به ترتیب descending از 14 به 1 است.

(۱۲ الف)

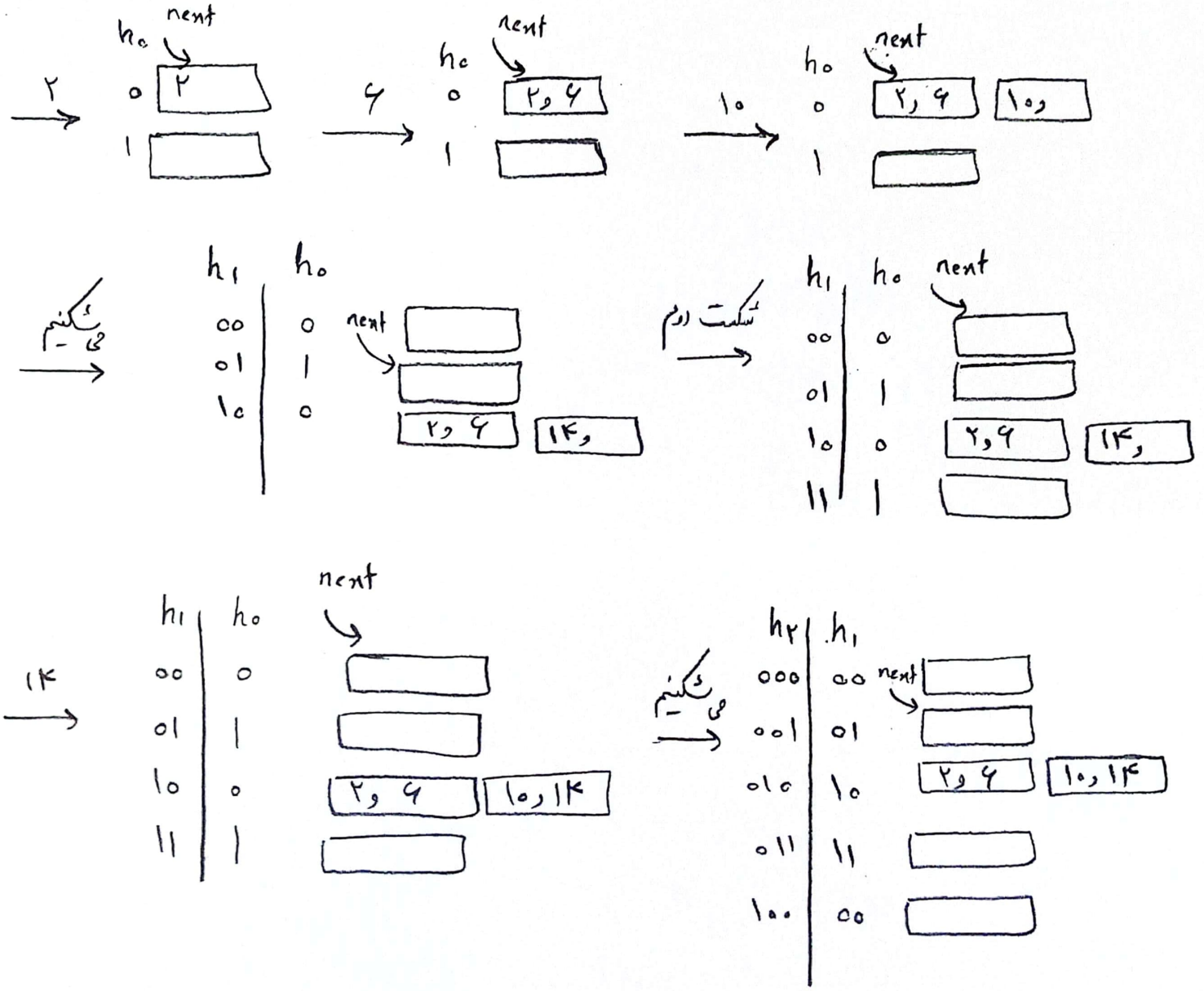

h0
next
10
26
42,
h1
00
01
10
11
شکست
شکست دوم
25
31
58
9
29
h2
000
001
010
011
100
ادامه دارد ...

ادامه ۲) الف)


h_2
h_1
000
00
001
01
010
10
011
11
100
00
101
01
next
25, 9
10, 26
42, 58
31
29

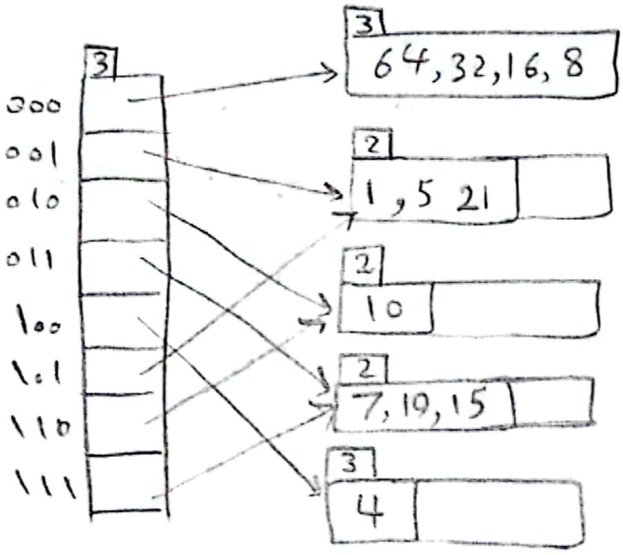
۲)

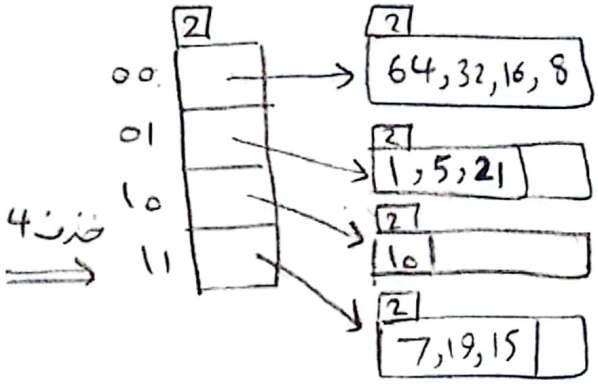
ب) به ترتیب عددهای ۲، ۶، ۱۰ و ۱۴ را insert می‌کنیم:



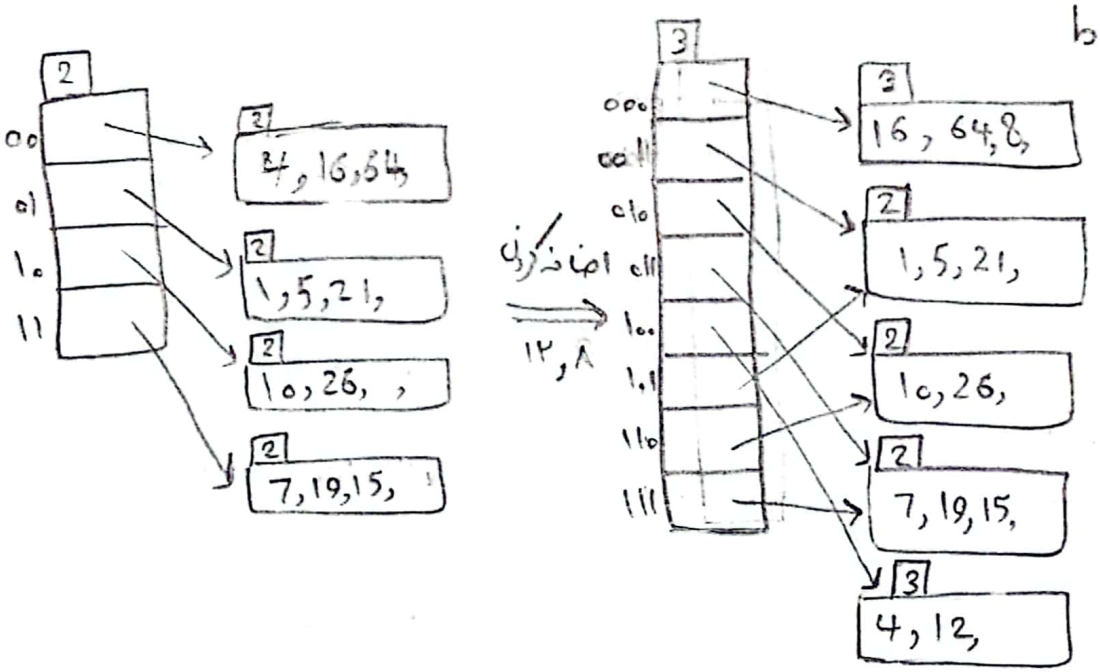
٣)

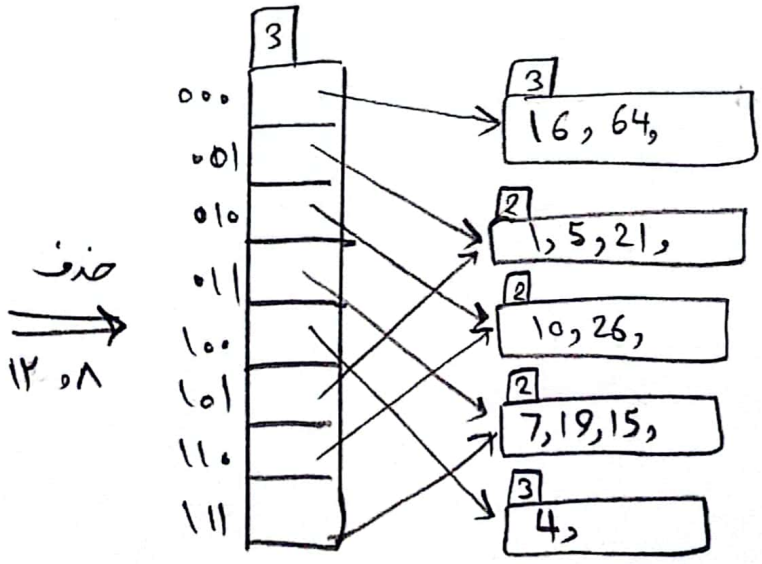
الف)


3
64,32,16,8
000
001
010
011
100
101
110
111
2
1,5 21
10
7,19,15
3
4


حذف 4
2
64,32,16,8
00
01
10
11
1,5,21
10
7,19,15

ب) $a = 12$, $b = 8$

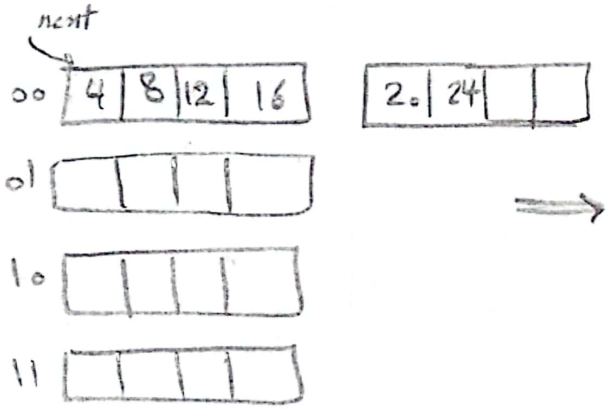

2
84,16,64
00
01
10
11
1,5,21,
10,26,
7,19,15,
اضافة 12,8
3
16,64,8,
000
001
010
011
100
101
110
111
1,5,21,
10,26,
7,19,15,
4,12,


حذف
12,8
3
16,64,
000
001
010
011
100
101
110
111
1,5,21,
10,26,
7,19,15,
4,

۱۴)

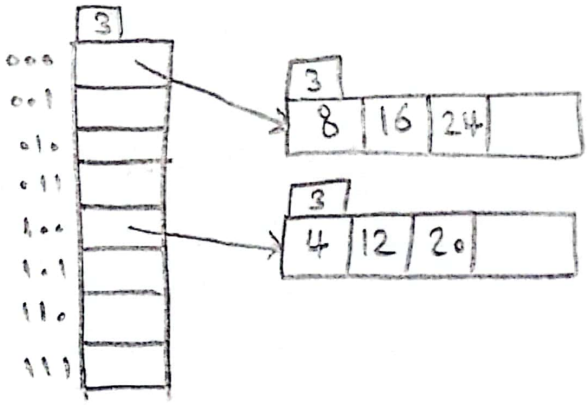
الف) اعداد ۴، ۸، ۱۲، ۱۶، ۲۰ و ۲۴ را به ترتیب insert می‌کنیم:

در linear hash به شکل زیر:



⟹ page آخر

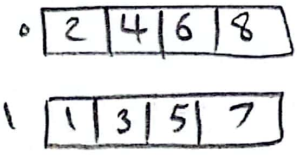
و در extendible hash به شکل زیر:



⟹ page آخر

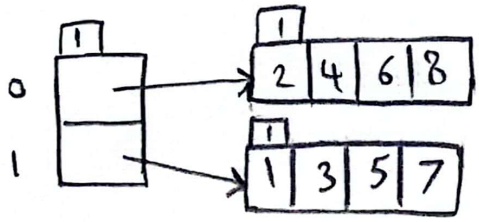
ب) اعداد ۱ تا ۸ را به ترتیب insert می‌کنیم:

در linear hash به شکل زیر:



⟹ page آخر 2

در extendible hash به شکل زیر:



⟹ page آخر 3

الف) یک IO . یعنی وقتی hash را حساب کردم و به باکت مورد نظر رسیدیم، آن page را داخل حافظه اصلی بیاوریم و رکوردهای آن page قابل دسترسی با یک IO هستن (اگر overflow page داشته باشیم، برای رکوردهای درون overflow page به بیش‌تر از یک IO نیاز است. اما کمترین تعداد IO، برای همین page اول از آن است که با یک IO خوانده می‌شود)

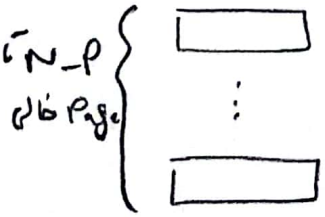
ب) کمترین میزان IO لازم، دوتا IO است و بهترین حالت. یک IO برای آوردن page مربوط به directory. بعد هش را حساب می‌کنیم و آن صفحه‌ای را که pointer بهش اشاره می‌کند، با یک IO دیگر به حافظه می‌آوریم. الان به record‌های آن page با دوتا IO، دست یافتیم.

ج) بدترین حالت این است هش تمام اعداد (رکوردها) که insert می‌کنیم، یکی شود و یک‌جا بیفتد و به گونه‌ای باشد که وقتی trigger شده و شکست رخ می‌دهد، هیچ رکوردی از آن به bucket دیگر نشود و همه رکوردها درهمان ردیف اول باشد و split شوند bucketهای مختلف. در این حالت به تعداد زیادی overflow page داریم که به ازای هر کدام، یک IO اضافی لازم داریم (نسبت به حالت الف). P تا از N تا رکورد درهمان Page اول هستند اما N-P تای بعدی در overflow page‌ها هستند. پس تعداد overflow page‌ها برابر است با $\lceil \frac{N-P}{P} \rceil$. درنتیجه تعداد دسترسی‌های لازم برابر است با $1 + \lceil \frac{N-P}{P} \rceil$

↓ اول Page ← تعداد page‌های سرریز ← حداکثر

از طرفی به ازای هر یک از N-P تا رکورد که به overflow اضافه می‌شود، یک bucket خالی درست می‌شود





پس تعداد کل فضای گرفته شده: $P \times \left(1 + \lceil \frac{N-P}{P} \rceil + (N-P)\right)$

↓ حجم هر Page ← تعداد کل Page‌ها

$$\text{space utilization} \Leftarrow \frac{N \leftarrow \text{کل رکوردها}}{P \times \left(1 + \lceil \frac{N-P}{P} \rceil + (N-P)\right)}$$

ادامه ۵)

۱) سوال از ما خواسته که شرایطی را بیان کنیم که در بدترین حالتش به $\lceil \frac{N}{P} \rceil$ دسترسی نیاز داریم (نکته: بدترین حالت رو بیان کنید سپس. این حالت که می‌گیم در بدترین حالت به $\lceil \frac{N}{P} \rceil$ تا دسترسی نیاز داره اما بدترین حالت دسترسی برای extendible hash ، $\lceil \frac{2^{N-P}}{P} \rceil + 1$ تا دسترسی و حتی بیشتر، که خیلی بیشتر از $\lceil \frac{N}{P} \rceil$ است.)

حال شرایطی را در نظر بگیرید که $N-P$ تا bucket داریم (مثلا دیگه باکت P تا رکورد، در یکی ۲ تا رکورد و در $N-P-2$ تا باکت دیگر هر کدام یک record باشه که در مجموع می‌شود $N-P$ تا باکت) پس حداقل $N-P$ تا Ptr نیاز داریم. حال شرایطی را در نظر بگیرید که $N-P$ ، توان از ۲ باشه (چون اگر نباشه، تعداد Ptr ها را باید ببریم به اولین توان ۲ی بزرگتر از اون) پس به Ptr های اضافه نیازی نیست و این $N-P$ تا Ptr در $\lceil \frac{N-P}{P} \rceil$ تا page بخش می‌شوند.

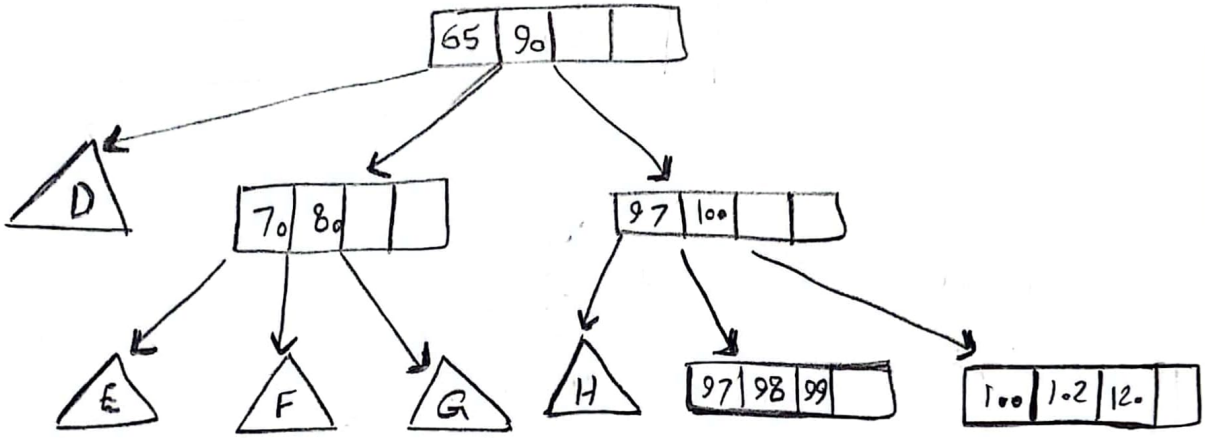
حال در بدترین حالت دسترسی، باید تمام این $\lceil \frac{N-P}{P} \rceil$ تا page مربوطه directory را بخوانیم تا در نهایت Ptr مربوط به آن صفحه‌ای که می‌خوایم را پیدا کنیم و بعد یک IO دیگه می‌زنیم تا به آن page (bucket) بریم. پس در نهایت تعداد کل دسترسی ها در بدترین حالت در این شرایط توصیف شده خاص برابرست با $\lceil \frac{N-P}{P} \rceil + 1 = \lceil \frac{N}{P} \rceil$

الف) درخت B+tree متوازن و balance است و همه برگ‌ها در یک سطح قرار دارند. پس عمق A باید ۱ باشد تا برگ‌ها با هم سطح با برگ‌های دیگر شود (یعنی یک لول node میانی و یک لول برگ است)

ب) حداکثر عدد ممکن در C باید کمتر از 37 باشد. پس می‌تواند 36 باشد و حداقل عدد ممکن در D باید بزرگتر مساوی 37 باشد پس خود 37 است. پس 36 برای C و 37 برای D

ج) حداقل به دو تا حذف نیاز داریم. 125 و 122 را حذف می‌کنیم و از آنجایی که تنها sibling آن‌ها، دو تا عدد دارد، پس تقع merge رخ می‌دهد و به خاطر همین، 120 در لایه بالاتر حذف و redistribution در عمق ۲ رخ می‌دهد و خواهیم داشت در نهایت: (زیر درخت ست چپ root و خود root تغییر نمی‌کند)

(در زیر درخت ست راست root داریم:)



د) در اسلاید که 122 و 125 را حذف کردیم، به ترتیب 99، 120 و 102 را حذف می‌کنیم. پس در مجموع 5 حذف داریم و خواهیم داشت

